

AHB to APB Bridge

Kailash.C

Contents

1	Introduction	2
2	Architecture	2
3	Block Diagram	3
4	AHB Slave Interface	4
4.1	Block Diagram	4
4.2	Functionality	5
4.3	Output Waveform	6
5	APB Controller	6
5.1	Block Diagram	6
5.2	Functionality	7
5.3	Finite State Machine	8
5.4	Output Waveform	8
6	Top Module	9
6.1	Verification of Top bridge	9
6.2	Output Waveform	9
7	Synthesis	11
8	Conclusion	11

1 Introduction

The AMBA (Advanced Microcontroller Bus Architecture) protocol is widely used in System-on-Chip (SoC) designs. The two main buses defined in the AMBA protocol are:

- Advanced High Performance Bus(AHB) : It is used for high performance, high clock frequency modules. It supports the connection of processors, on chip memories and external memory interfaces.
- Advanced Peripheral Bus(APB) : It is used for low power peripherals. This bus is mainly optimized for minimal power consumption. It also has a much simpler interface as compared to AHB.

The important terms used in this protocol are:

- Bus Cycle : It is the basic unit of one bus clock period. For AHB and APB, It is defined from rising edge to rising edge transitions.
- Bus Transfer : AHB bus transfer is defined as a read or write operation of a data object which takes one or more than one bus cycles. APB bus transfer is defined as a read or write operation of a data object which always takes two bus cycles.
- Burst Operation : It is defined as multiple data transfers initiated by a master. This operation is not supported on APB.

2 Architecture

The AHB-to-APB Bridge is designed to interface the high-performance AHB with the low-power APB. Since AHB and APB operate using different protocols and timing requirements, a bridge is required to facilitate communication between AHB masters and APB peripherals.

The design mainly consists of two major blocks:

1. **AHB Slave Interface**
2. **APB Controller**

The AHB Slave Interface receives address, control and data signals from the AHB master. It performs address decoding, generates the valid transfer signal and pipelines the address and data information required for APB transactions. Based on the received address, the interface also generates the peripheral select signal (PSELx) used for APB slave selection.

The APB Controller is implemented using a Finite State Machine (FSM). It receives the decoded control signals from the AHB Slave Interface and generates the APB control signals such as PADDR, PWRITE, PENABLE and PSELx.

During a write operation, the bridge captures the address and data from the AHB side and generates the corresponding APB write transaction. During a read operation, data received from the APB peripheral is forwarded back to the AHB master through

the bridge. The design supports single as well as burst transfers and enables efficient communication between high-speed AHB masters and low-speed APB peripherals.

In this project, the bridge is designed for one master and multiple slaves purpose.

3 Block Diagram

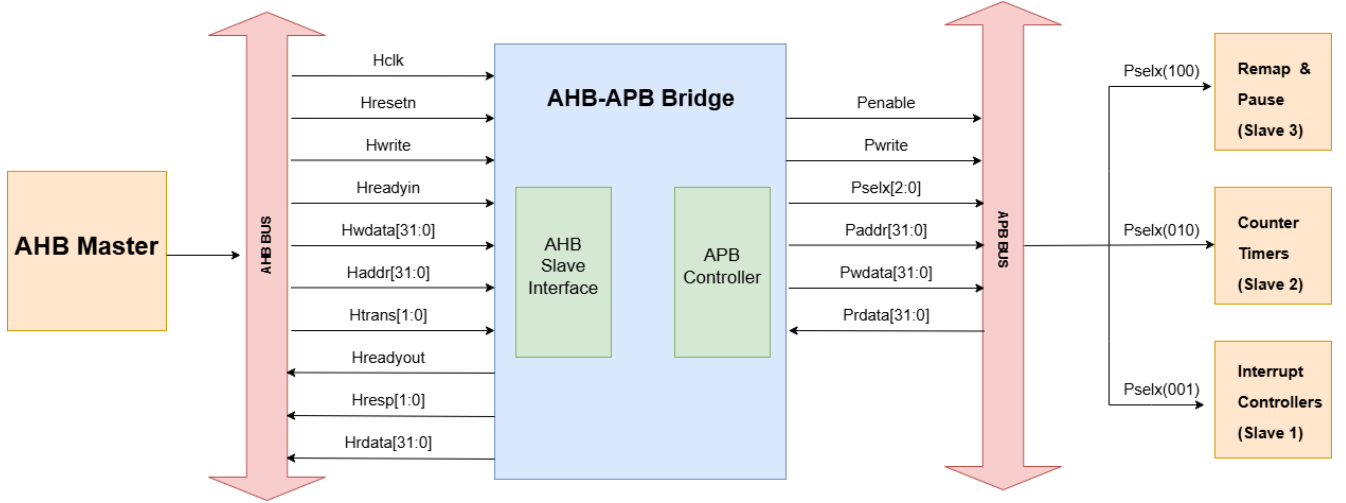


Figure 1: Block Diagram of the AHB-to-APB Bridge

The above figure illustrates the overall architecture of the proposed AHB-to-APB Bridge. Below is the signal description from this block diagram.

Table 1: AHB Signal Description

Signal	Width	Description
HCLK	1	System clock used for all AHB transactions.
HRESETn	1	Active-low reset signal.
HADDR	32	Address bus carrying the transaction address.
HWDATA	32	Write data bus from the AHB master.
HRDATA	32	Read data bus returned to the AHB master.
HWRITE	1	Indicates read or write operation.
HTRANS	2	Specifies the type of AHB transfer.
HREADYIN	1	Indicates that the previous transfer has completed.
HREADYOUT	1	Generated by the bridge to indicate transfer completion.
HRESP	2	Response signal indicating transaction status(OKAY, ERROR, RETRY and SPLIT).

Table 2: APB Signal Description

Signal	Width	Description
PADDR	32	APB address bus.
PWDATA	32	APB write data bus.
PRDATA	32	APB read data bus.
PWRITE	1	Indicates APB read or write operation.
PENABLE	1	Enables the APB transfer.
PSELx	3	Peripheral select signal used to select an APB slave(Uses one hot encoding).

4 AHB Slave Interface

4.1 Block Diagram

The AHB Slave Interface forms the front-end of the AHB-to-APB Bridge. It receives address, data and control information from the AHB master and prepares the required signals for the APB Controller. Below is the block diagram:

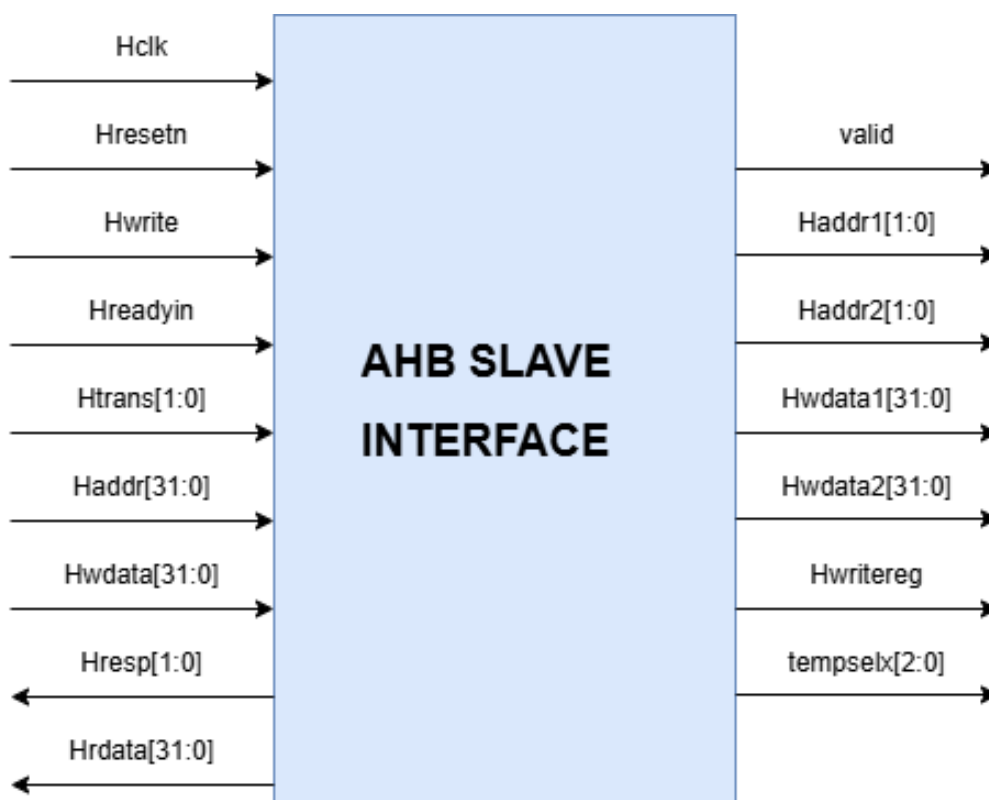


Figure 2: AHB Slave Interface Block Diagram

4.2 Functionality

The AHB Slave Interface performs the following operations:

1. Generation of the valid transfer signal.
2. Address pipelining using `haddr1_o` and `haddr2_o`.
3. Data pipelining using `hwdata1_o` and `hwdata2_o`.
4. Write control signal synchronization using `hwritereg_o` and `hwritereg1_o`.
5. Address decoding for APB peripheral selection.
6. Read data forwarding from APB to AHB.

The signal `valid_o` is asserted only when the following conditions are satisfied:

- `HREADYIN` is asserted.
- The address lies within the APB address range (0x8000_0000 to 0x8C00_0000).
- The transfer type corresponds to `NONSEQ` (2'b10) or `SEQ` (2'b11).

The module also generates the slave select signal `tempselx_o` using one-hot encoding. Based on the decoded address range, one of the APB slaves is selected as shown below in the Table.

Table 3: Address Decoder Logic

Address Range	<code>tempselx_o</code>
0x8000_0000 - 0x8400_0000	001
0x8400_0000 - 0x8800_0000	010
0x8800_0000 - 0x8C00_0000	100
Others	000

Two-stage pipelining is implemented for both address and write data signals. The registers `haddr1_o` and `haddr2_o` store one-cycle and two-cycle delayed versions of the incoming address respectively. Similarly, `hwdata1_o` and `hwdata2_o` store delayed versions of the write data. These pipelined signals are utilized by the APB Controller during burst transfers.

The write control signal `HWRITE` is also delayed through two registers (`hwritereg_o` and `hwritereg1_o`) to maintain synchronization with the pipelined address and data paths.

For read transactions, the APB read data signal `PRDATA` is forwarded to the AHB side through `HRDATA`. The response signal `HRESP` is permanently assigned the value 2'b00, indicating an **OKAY** response for all transactions.

4.3 Output Waveform

Below Figure shows the simulation results of the AHB Slave Interface.

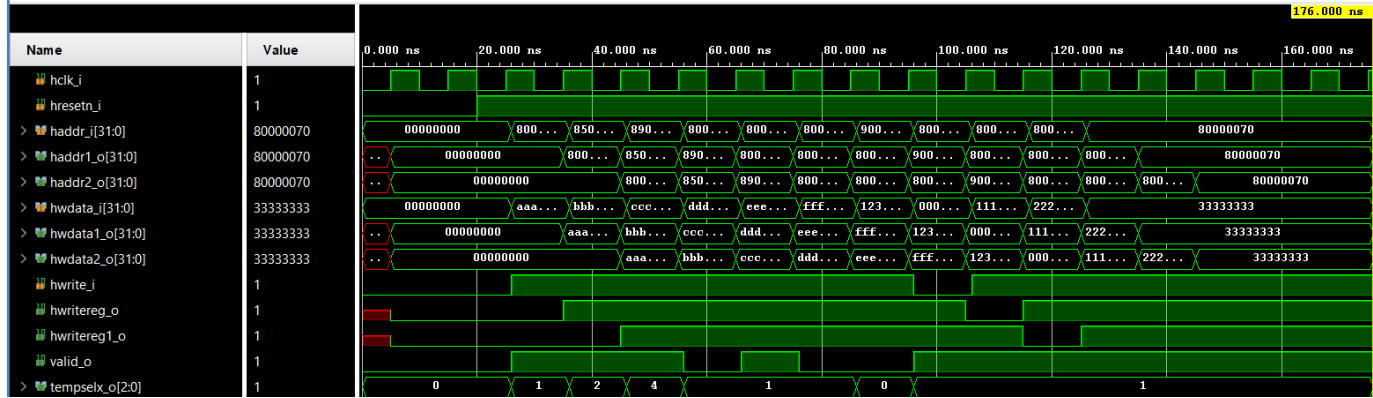


Figure 3: AHB Slave Interface Output Waveform

From the waveform, it can be observed that the address pipeline registers `haddr1_o` and `haddr2_o` follow the input address `haddr_i` with one-clock-cycle and two-clock-cycle delays respectively. Similarly, the write data pipeline registers `hwdata1_o` and `hwdata2_o` follow the input write data `hwdata_i` with corresponding delays.

The signals `hwritereg_o` and `hwritereg1_o` are observed to be delayed versions of `hwrite_i`, thereby maintaining synchronization between the address, data and control paths.

The signal `valid_o` is asserted only during valid AHB transactions. Furthermore, the decoder output `tempselx_o` changes according to the address range and correctly selects the target APB slave using one-hot encoding.

The simulation results verify the correct operation of the address pipeline, data pipeline, write control synchronization, transfer validation logic and address decoder implemented within the AHB Slave Interface.

5 APB Controller

5.1 Block Diagram

The APB Controller forms the control unit of the AHB-to-APB Bridge. It receives the pipelined address, data and control signals generated by the AHB Slave Interface and converts them into APB-compliant transactions. Below Figure shows the block diagram of the implemented APB Controller.

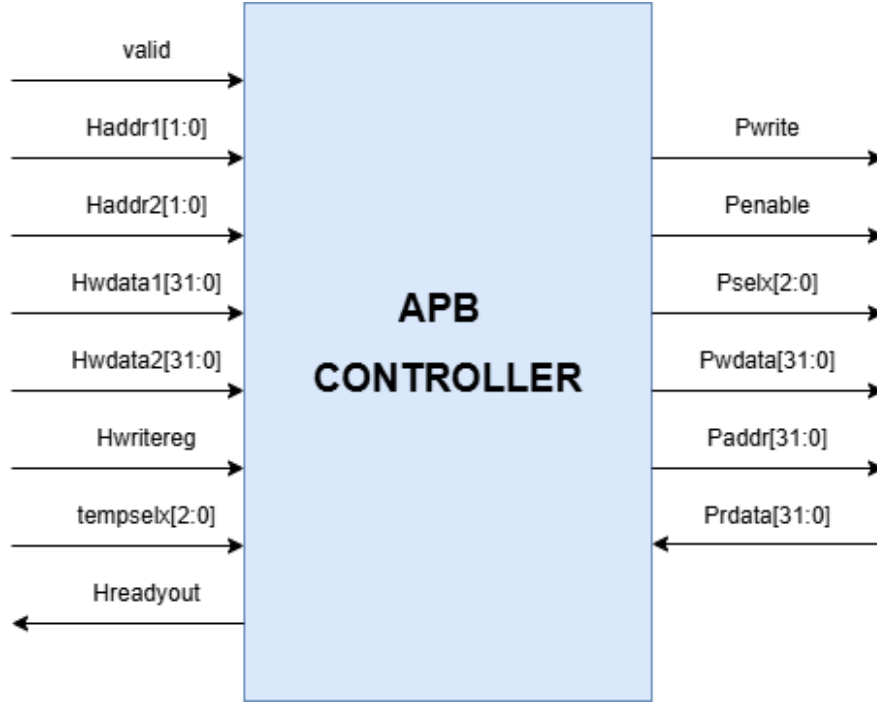


Figure 4: APB Controller Block Diagram

5.2 Functionality

The APB Controller is implemented using a Finite State Machine (FSM) and performs the following operations:

1. Generation of APB control signals.
2. Conversion of AHB transactions into APB transactions.
3. Generation of APB address and write data signals.
4. Generation of peripheral select signals.
5. Generation of `hreadyout_o` for AHB synchronization.
6. Support for single and burst write transactions.
7. Support for APB read transactions.

The controller receives the signals `valid_o`, `hwrite_i`, `hwritereg_i`, `haddr1_i`, `haddr2_i`, `hwdata1_i`, `hwdata2_i` and `tempselex_i` from the AHB Slave Interface and generates the corresponding APB outputs.

The controller generates the APB signals `paddr_o`, `pdata_o`, `pwrite_o`, `penable_o` and `pselex_o`. These signals are responsible for carrying out APB setup and enable phases.

5.3 Finite State Machine

The APB Controller FSM consists of eight states:

Table 4: APB Controller FSM States

State	Function
ST_IDLE	Waits for a valid AHB transaction.
ST_WWAIT	Wait state before initiating a write transfer.
ST_WRITE	Performs APB write operation.
ST_WRITEP	Performs pipelined write operation.
ST_WENABLEP	Enable phase of a pipelined write transfer.
ST_WENABLE	Enable phase of a write transfer.
ST_READ	Performs APB read operation.
ST_RENABLE	Enable phase of a read transfer.

The FSM ensures proper synchronization between AHB and APB protocols while supporting burst write transactions and read operations.

5.4 Output Waveform

Below Figure shows the simulation results of the APB Controller during a Burst Write WRAP4 transaction:

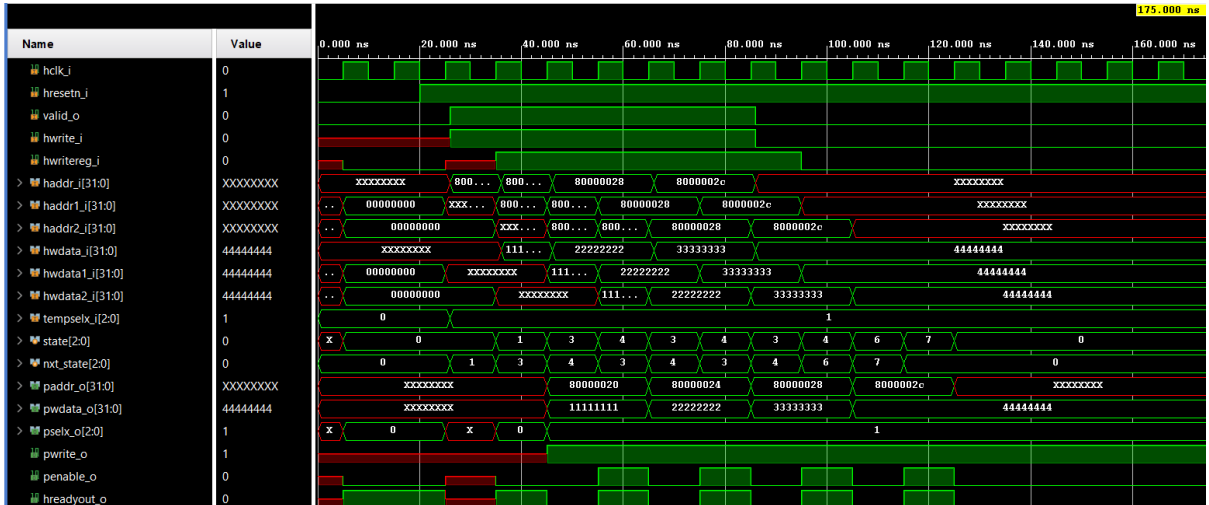


Figure 5: APB Controller Output Waveform

From the waveform, it can be observed that the FSM transitions through the states ST_IDLE, ST_WWAIT, ST_WRITEP and ST_WENABLEP during the burst write transaction. The signals **state** and **nxt_state** clearly illustrate the progression of the controller through the APB transfer sequence.

The address signal **paddr_o** follows the pipelined AHB addresses and generates the corresponding APB addresses. Similarly, the signal **pwddata_o** follows the pipelined write data and transfers it to the selected APB peripheral.

The signal `pselect_o` is asserted to select the target APB slave, while `pwrite_o` remains high throughout the write transaction, indicating an APB write operation.

The signal `penable_o` is turned on during the enable phase of each APB transfer.

The signal `hreadyout_o` is generated by the controller to synchronize the AHB and APB buses. It is turned off whenever additional APB cycles are required and turned on once the transfer can proceed.

The simulation results verify the correct operation of the FSM, APB signal generation logic and AHB-to-APB protocol conversion implemented within the APB Controller.

6 Top Module

6.1 Verification of Top bridge

To verify the functionality of the complete AHB-to-APB Bridge, an AHB Master model and an APB Interface model were developed for simulation purposes.

The AHB Master module generates the required AHB transactions and provides address, data and control signals to the bridge. In this project, a Burst Write WRAP4 transaction was used to verify the operation of the bridge. The master generates the address sequence, write data values and transfer control signals required for the burst transaction.

The APB Interface module simulates the APB slave peripherals. It receives the APB control signals generated by the bridge and responds appropriately to read and write operations. For read transactions, the module generates sample read data values which are returned to the AHB side through the bridge.

6.2 Output Waveform

Below Figure shows the simulation results of the complete AHB-to-APB Bridge during a Burst Write WRAP4 transaction.

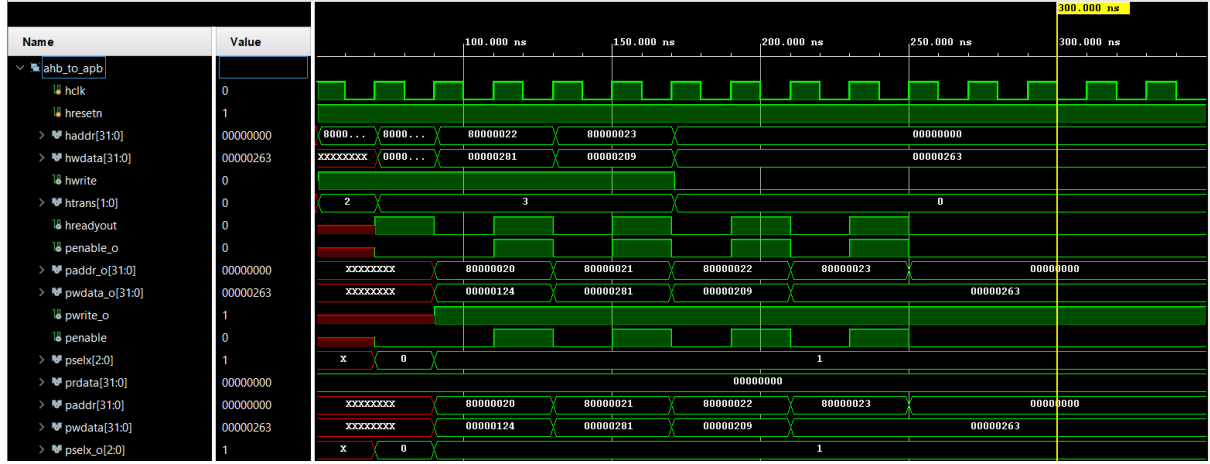


Figure 6: Top Module Output Waveform

The waveform illustrates the complete protocol conversion from the AHB bus to the APB bus. The AHB Master initiates a Burst Write WRAP4 transaction by turning on **hwrite** and generating the address sequence through **haddr**. The transfer type signal **htrans** transitions from NONSEQ (2'b10) to SEQ (2'b11), indicating the beginning and continuation of the burst transfer.

The AHB write data provided on **hwrite** is accepted by the bridge and processed through the address and data pipelines implemented in the AHB Slave Interface. The APB Controller subsequently generates the corresponding APB transactions.

It can be observed that the APB address signal **paddr** follows the incoming AHB addresses and generates the sequence:

$$0x80000020 \rightarrow 0x80000021 \rightarrow 0x80000022 \rightarrow 0x80000023$$

Similarly, the APB write data signal **pwrite** follows the data received from the AHB side and transfers the values to the selected APB peripheral.

The signal **pwrite** remains on throughout the write transaction, indicating that all transfers correspond to APB write operations. The signal **penable** is on during the enable phase of each APB transfer, thereby implementing the two-phase APB protocol consisting of a setup phase followed by an enable phase.

The peripheral select signal **pselx** is generated using one-hot encoding and correctly selects the target APB slave based on the decoded address range.

The signal **hreadyout** is generated by the bridge to synchronize the AHB and APB buses. Whenever additional APB cycles are required, **hreadyout** is turned off, thereby introducing wait states on the AHB side. Once the APB transfer is completed, **hreadyout** is again turned on, allowing the next transfer to proceed.

The simulation results confirm that the bridge successfully converts AHB burst write transactions into corresponding APB write transactions while maintaining proper synchronization between the two protocols.

7 Synthesis

The AHB-to-APB Bridge was synthesized using Vivado. Below is the post-synthesis resource utilization of the design:

Resource	Utilization	Available	Utilization %
LUT	56	20800	0.27
FF	245	41600	0.59

Figure 7: Utilization Report

8 Conclusion

In this project, an AHB-to-APB Bridge was successfully designed and verified using Verilog HDL. The functionality of each sub-block was verified individually through simulation waveforms. The AHB Slave Interface was verified for address pipelining, data pipelining, write control synchronization and address decoding. The APB Controller was verified for correct state transitions and generation of APB control signals. The complete bridge was then verified using a Burst Write WRAP4 transaction. The simulation results demonstrated successful protocol conversion between the AHB and APB buses.